Coherence assessment of shared subgraph patterns of articles based on graph attention networks

Ya Zhou
School of Computer and Information Security
Guilin University of Electronic Science and Technology
Guilin, China
ccyzhou@guet.edu.cn

Yongqiang Hu
School of Computer and Information Security
Guilin University of Electronic Science and Technology
Guilin, China
hu_yong_qiang@163.com

2024 4th International Conference on Computer Science and Blockchain (CCSB) | 979-8-3503-5650-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/CCSB63463.2024.10735478

***Abstract*-Text coherence is crucial for ensuring both the quality and comprehensibility of written material and is vital for various natural language processing tasks and applications. While existing research in feature engineering and neural networks has achieved commendable results, these studies have predominantly focused on content within individual documents, neglecting the structural similarities that may exist between different texts. This study explores a coherence model based on graph attention networks, aiming to capture structural similarities across texts and assess their coherence based on shared subgraph patterns. The approach involves first constructing a graph structure for each text, grounded in rhetorical structure theory and semantic relations, and then identifying different subgraph patterns. Subsequently, a document-pattern graph is built, representing shared subgraph patterns. Finally, the graph attention network is employed to model this graph. Experimental results indicate that this method outperforms baseline models in discourse coherence evaluation tasks, demonstrating notable advancements.**

Keywords-text coherence; rhetorical structure theory; subgraph patterns; graph attention networks

I. INTRODUCTION

Coherence is an essential attribute of high-quality text, referring to the degree to which sentences are logically connected and the overall organization of a document. In a coherent text, each part is logically linked to the preceding one, enhancing the clarity of the text and making it easier for readers to understand. Coherence is a critical aspect of text quality and is crucial for a variety of Natural Language Processing (NLP) tasks, such as summarization[1], machine translation[2], automated essay scoring[3] and text generation[16].

Early research focused on feature engineering-based methods, often inspired by linguistic theories. For example, the Entity-grid method, influenced by Centering Theory, models local coherence by considering the co-occurrence of entities in adjacent sentences. Similarly, approaches based on rhetorical structure theory (RST) [4] are inspired by discourse theory and model local coherence through the discourse relations between sentences. Other methods include those based on co-occurrence structural syntactic patterns, semantic relatedness, and thematic models.

Recently, neural coherence models have been employed to address coherence issues, owing to their advantages in automatic feature extraction, powerful learning capabilities, and broad applicability. These models learn representations of words, sentences, paragraphs, and even entire texts, extracting key features highlighted in linguistic theories for use in learning tasks. Their performance significantly surpasses that of earlier feature engineering-based approaches. Not only do they utilize deep learning models, such as self-attention mechanisms, to capture logical and semantic relationships between sentences, but they also incorporate concepts from linguistics, such as RST, into model design to enhance the understanding and evaluation of text coherence.

However, both feature engineering-based methods and neural network-based approaches share a common limitation: they focus solely on extracting linguistic features from individual texts, neglecting the structural similarities that exist between texts. Theoretically, texts with similar structural connections should exhibit similar levels of coherence, which can serve as prior knowledge in coherence modeling. This paper proposes a coherence model that captures structural similarities between texts, leveraging RST, semantic relations, subgraph frequency, and shared subgraph patterns. The approach begins by parsing texts using RST and representing the connections between elementary discourse units (EDUs) as an RST graph, where each EDU—a basic unit in RST—serves as a node. Next, nouns within each EDU are identified, and semantically related nodes are connected. Subgraph patterns within the generated graph are then identified, and their frequencies are calculated. A document-pattern graph is constructed with texts and subgraph patterns as nodes, which is subsequently used for learning and training.

The proposed method was evaluated on a discourse coherence task. Experimental results demonstrate that this approach significantly outperforms baseline models that do not consider structural similarities between documents, as well as those that do not utilize RST, achieving new advancements in this task.

II. RELATED WORK

Feature engineering based approaches: Most research on feature engineering-based approaches has focused on entity-gird. Barzilay and Lapata proposed a novel framework for representing and measuring local coherence based on entity co-references, known as the Entity-Grid model, which is grounded in Centering Theory to capture patterns in entity distribution within text. A large body of work has utilized and extended the entity-grid approach, incorporating additional entity information, integrating RST. However, the main drawbacks of

Authorized licensed use limited to: UNIVERSITY OF TENNESSEE LIBRARIES. Downloaded on September 23,2025 at 04:00:42 UTC from IEEE Xplore. Restrictions apply.

the entity grid and its extensions are that the use of discrete representations leads to the curse of dimensionality, and the separation of feature engineering from the prediction task limits the model's learning capabilities.

Another study, inspired by discourse theories such as RST, utilizes discourse relations between sentences to predict coherence. Guz et al. [18] proposed an RST-Recursive model to investigate the usefulness of silver-standard parsed RST features in neural coherence classification. Adewoyin et al. proposed a framework for controlling the discourse structure, semantics, and themes of generated texts using RST to improve the articulation and coherence of long generated texts.

Deep Learning Methods: Recently, a variety of deep learning methods have been proposed. Li and Jurafsky[5] explored discriminative models for distinguishing concatenated text, as well as generative models for creating such text, capable of capturing potential discourse dependencies between sentences. Mesgar and Strube[6] utilized subgraph frequencies in lexical coherence graphs to encode coherence features, achieving higher accuracy by capturing more structural relationships through larger subgraphs. Lai and Tetreault[7] introduced SENTAVG, an LSTM-based neural network model that disregards sentence order, and also published a three-layer LSTM neural architecture called ParSeq. Moon et al.[8] presented a unified coherence model using a bidirectional long short-term memory network (bi-LSTM) to encode each sentence as a vector representation, integrating sentence syntax, discourse relations, attention, and topic structure into a single framework. Jeon and Strube[9] computed coherence based on entities using a large language model, addressing the issue of pre-trained language models that encode input documents at the sub-word level—an approach that may not align with linguistic coherence. Abhishek et al.[10] proposed four different architectures based on the Transformer model to assess text coherence. Liu et al.[11] introduced a method for connecting structurally similar documents based on shared subgraphs, exploring the effectiveness of structural similarity information between documents in coherence modeling and addressing the limitation of neural network models that focus solely on single documents.

Graph-based approaches to natural language processing: graph neural networks (GNNs) are a class of neural networks that operate naturally on graphs. Applying GNNs to NLP tasks has garnered significant attention. Yasunaga et al.[12] proposed a GCN-based multi-document summarization system. Wang et al. introduced a novel adaptive labeled segmentation graph convolutional network for aspect-level sentiment analysis. Huang and Carley developed a goal-dependent graph attention network (TD-GAT) for aspect-level sentiment categorization; Liang et al.[13] proposed a bi-grammatical aware graph attention network(BiSyn-GAT+) framework, which leverages the grammatical information of sentence composition trees to model sentiment-aware contexts for each aspect and the sentiment relations across aspects.

III. Methods

The proposed model first constructs a graph structure for each document based on RST and semantic relations, then identifies various subgraph patterns, and finally constructs a document-pattern graph, which is modeled using a Graph Attention Network (GAT) [17]. The specific process is illustrated in Figure 1: First, the text is parsed using RST to progressively generate an RST tree and an RST graph. Then, semantic similarity is calculated with EDUs as nodes, and semantically related nodes are connected to form subgraphs that include both RST and semantic relations. Next, k-node subgraphs are identified within each graph, and their frequencies are calculated. A document-pattern graph is then constructed with the text and the identified subgraph patterns as nodes, based on the co-occurrence relationship between the documents and subgraph patterns. Finally, the GAT encoder is used to learn and predict the labels of the texts.

A. Structural Graph Generation Module

To model the structural similarity between documents, the proposed model first identifies the structure of each document and represents the text as a directed graph. The following explains the process of generating an RST tree from the text and converting the RST tree into an RST graph. According to RST, the structure of a text can be represented as an RST tree, with the Elementary Discourse Unit (EDU) as the basic unit. In the RST tree, EDUs are categorized as either "Nucleus (N)" or "Satellite (S)," with the nucleus being more important than the satellite.

Following the method by Y. Ji [14], RST discourse parsing is conducted through representation learning (as shown in Figure 1). This approach maps surface features into a latent space, thereby simplifying the analysis of RST texts. However, in some cases, the RST tree may not sufficiently capture the complex structural relationships between text elements, as it does not always explicitly define parent-child relationships between text units. To address this, the model converts the RST tree into an RST graph following the method of Hirao [15] (illustrated in Figure 2), while preserving the RST structure and incorporating semantic relations to enrich the representation of the text's structure. If the nouns in two EDUs exhibit a high degree of semantic similarity, these EDUs are considered semantically related. In this model, although entities are often used for semantic modeling between sentences, nouns have proven more effective in representing semantic relations. Therefore, the model first identifies all nouns within each EDU and then calculates the cosine similarity between nouns in every pair of EDUs, selecting the highest value as the similarity score between the two EDUs.

By integrating both RST and semantic relations, the model ultimately generates a comprehensive structural graph of the text, which more accurately reflects the document's structural and semantic information.

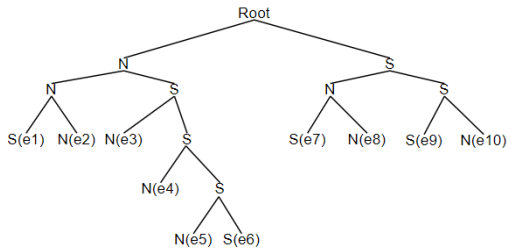

Figure 1: An example of an RST tree

Authorized licensed use limited to: UNIVERSITY OF TENNESSEE LIBRARIES. Downloaded on September 23,2025 at 04:00:42 UTC from IEEE Xplore. Restrictions apply.

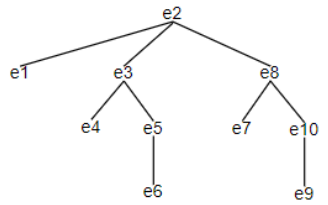

Figure 2: The RST graph generated based on Figure 2

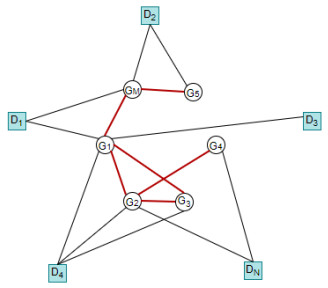

Figure 3: An example of document-pattern graph. For simplicity, we only show five documents and six subgraphs, black lines indicate edges between documents and patterns, red lines indicate edges between patterns

B. Document-Pattern Graph

After generating the structural graph of the text, the model identifies all k-node subgraphs and calculates the frequency of these subgraphs. The identified subgraphs represent the subgraph patterns extracted by the model. Subsequently, a document-pattern graph is constructed with both subgraph patterns and texts as nodes. Figure 3 illustrates the constructed document-pattern graph.

This graph contains two types of nodes: document nodes and subgraph nodes. The number of document nodes corresponds to the number of texts in the training set (N), while the number of subgraph nodes corresponds to the number of distinct k-node subgraph types identified (M).

The document-pattern graph includes two types of edges:

1. Document-Subgraph Edges: These edges connect document nodes to subgraph nodes, indicating that a particular subgraph pattern is present in a document. The weight of these edges can be set based on the frequency of the subgraph pattern in the document, reflecting the strength of the association between the subgraph pattern and the document. The weight of these edges is defined as:

$$A_{i,j} = \frac{f_{ij}}{NF_i} \cdot log \frac{N}{n_{doc}(g_i)} \tag{1}$$

where f_{ij} is the frequency of subgraph g_j in document d_i, NF_i is the total number of subgraphs in document d_i, and $n_{doc}(g_j)$ is the number of documents containing subgraph g_j.

2. Subgraph-Subgraph Edges: These edges connect different subgraph nodes, indicating the co-occurrence relationship between these subgraph patterns across different documents. This type of edge is used to capture the potential associations and similarities between subgraph patterns. The weight of these edges is given by:

$$A_{i,j} = log \frac{p(i,j)}{p(i)p(j)} \tag{2}$$

where $p(i,j)$ is the probability that subgraphs g_i and g_j co-occur, and $p(i)$and $p(j)$ are their individual probabilities. To focus on positive correlations, negative PMI values are set to 0.

C. GAT Encoder Module

The proposed model utilizes GAT to encode the constructed document-pattern graph. GAT operates directly on graph-structured data, assigning different weights to various nodes within the neighborhood, thereby effectively learning node representations based on the graph's connection patterns and features. The attention mechanism in GAT is particularly advantageous in handling variable-sized inputs and focusing on the most relevant parts, enabling it to dynamically adjust attention towards different neighboring nodes and better capture task-related information.

Linear Transformation: The matrix A generated from the document-pattern graph is a $(N+M) \times (N+M)$ matrix, where w_{ij} represents the weight between node v_i and node v_j. The feature of node v_i can be denoted as a_{i}. At each attention layer, a_{i} undergoes a linear transformation to learn and weight the information relevant to itself, according to the following equation:

$$h_i' = \mathrm{W}h_i \tag{3}$$

where W is a trainable weight matrix, initialized randomly, h_{i} is the input feature of node v_i, and h_i' is the transformed feature vector.

Calculating Attention Scores: The document-pattern graph is represented as a weighted adjacency matrix, where the model uses edge weights to adjust the calculation of attention coefficients. Initially, the unnormalized attention scores between v_i and v_j are computed as follows:

$$e_{ij} = LeakyReLU\ (a^T[h_i'||h_j']) \cdot w_{ij} \tag{4}$$

Here, LeakyReLU is a non-linear activation function, and the attention mechanism a is a single-layer feedforward neural network. a^T denotes the transpose of a, represented as a row vector, || indicates the vector concatenation operation, and $w_{i,j}$ is the edge weight between nodes v_i and v_j.

Next, the attention scores of all neighboring nodes are normalized using a softmax function (as illustrated in Figure 4). The softmax function transforms the original attention scores into non-negative values and normalizes them so that the sum of the attention weights of all neighboring nodes equals 1, resulting in the normalized attention coefficient a_{ij}. Considering all neighboring nodes j of node i, the normalized attention coefficient is given by:

$$a_{ij} = \frac{exp(e_{ij})}{\sum_{k \in N_i} exp(e_{ik})} \tag{5}$$

where the numerator $exp(e_{ij})$ computes the exponential of the attention score between the two nodes, and the denominator computes the sum of the exponentials of the attention scores of all neighboring nodes of i.

Authorized licensed use limited to: UNIVERSITY OF TENNESSEE LIBRARIES. Downloaded on September 23,2025 at 04:00:42 UTC from IEEE Xplore. Restrictions apply.

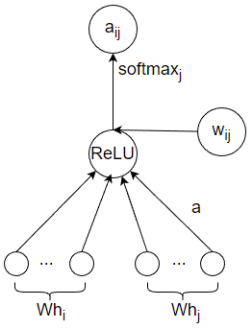

Figure 4: The attention mechanism used by the model presented in this paper

After obtaining the linear transformation of the node features and the attention scores, the neighboring node features are aggregated based on the normalized attention coefficients a_{ij}:

$$h_i' = \sigma(\sum_{j \in N_i} a_{ij} h_j') \tag{6}$$

Experiments have shown that using a multi-head attention mechanism yields better results. Applying equation (6) with K different attention mechanisms and concatenating the output features in sequence, followed by averaging, results in the final output feature for each node:

$$h_i' = \sigma(\frac{1}{K}\sum_{k=1}^{K} \sum_{j \in N_i} a_{i,j}^k h_j') \tag{7}$$

where $a_{i,j}^k$ is the attention coefficient computed by the k-th attention mechanism. During evaluation, each document from the test corpus is added to the document-pattern graph, the features of the new graph are encoded, and the label of the document is predicted.

IV. Experiment

A. Datasets and assessment indicators

The Grammarly Corpus of Discourse Coherence (GCDC) [7] is a newly released dataset of real-world texts that can be written by ordinary people, contains texts from four domains: online forum posts from Yahoo, emails from Hillary Clinton's office, emails from Enron, and business reviews from Yelp, each containing 1000 documents for training and 200 documents for testing. Each document is assigned discrete coherence labels of low coherence (1), medium coherence (2), and high coherence (3), with the number of word categories and the total number of words shown in Table 1. The accuracy metric is used in this paper to evaluate the model's performance.

Table 1: Type and token counts in each domain

	Clinton	Enron	Yahoo	Yelp
types	16654	13694	13235	12201
tokens	220115	223347	213852	189444

B. Baseline

To demonstrate that our model does improve evaluation performance compared to other methods in the field of text coherence analysis, we compared our model with the following models:

LGG[6]: The Lexical Coherence Graph is a model that represents inter-sentence lexical relationships using a graph structure. The frequency of subgraphs (coherence patterns) within this graph captures the coherence of sentence nodes. The coherence of a text is encoded by the vector of these frequencies.

PARSEQ[7]: This model consists of three stacked LSTMs that encode sentence vectors, paragraph vectors, and document vectors respectively. The document vector is used to generate a distribution of coherence labels via a hidden layer and softmax function.

Unified coherence[8]: This model combines both local and global coherence models, integrating sentence syntax, discourse relations, attention mechanisms, and topic structures into a single framework.

RST-Recursive[18]: This model uses silver-standard parsed RST (Rhetorical Structure Theory) features for neural coherence classification.

MTL with T [10]: This approach proposes four different architectures based on the Transformer model to evaluate text coherence, with the best performance observed in the MTL with Transformer model.

GCN-based[11]: This model proposes a method that connects structurally similar documents based on shared subgraph structures and models these connections using a GCN. The study explores the effectiveness of inter-document structural similarity information in coherence modeling.

C. Experimental setup

We use the Pytorch library to implement the model. We use Y. Ji's method[14] for RST discourse parsing using representation learning to convert documents into trees in RST format, Hirao's method[15] for converting RST trees into RST graphs using rules for dependency-based discourse trees, the Stanza toolkit to identify nouns in each segmented EDU, and the GloVe pre-trained embeddings to compute the similarity between nouns, and we set the similarity threshold δ to 0.60, which has been experimentally demonstrated to yield good results.

For the construction of the subgraph set, we use a 6-node subgraph as the base unit of the task and limit the maximum sentence distance w to 8. Our approach employs a three-layer GAT with ReLU as the activation function. A 10-fold cross-validation is performed for the GCDC training dataset. The learning rate was set to 0.01 and the task applied Dropout with an initial learning rate of 0.5. We trained 200 epochs on the GCDC dataset.

D. Experimental results

Table 2 shows the experimental results on the GCDC dataset. The main findings are as follows: The proposed models based on GCN and GAT achieved high accuracy across all four datasets, with the GAT model outperforming the GCN model. Notably, models that utilised subgraph patterns (LGG, GCN-based, and the proposed model) exhibited higher accuracy than those that did not incorporate subgraph patterns. Encoding with only RST or subgraph patterns did not yield the best results. The

Authorized licensed use limited to: UNIVERSITY OF TENNESSEE LIBRARIES. Downloaded on September 23,2025 at 04:00:42 UTC from IEEE Xplore. Restrictions apply.

Unified Coherence model, which integrates multiple information sources for encoding, showed lower overall performance, as its ability to extract coherent patterns was less effective. While PARSEQ, using a three-layer LSTM, was effective at encoding words, sentences, and texts, it performed poorly in coherence expression, likely due to its neglect of common coherence patterns across documents. The MTL with T model, employing multitask learning and Transformers, was better at capturing text coherence features, yet its effectiveness remained limited.

The GAT-based model achieved the highest accuracy, indicating that focusing on relevant information through the attention mechanism can significantly improve performance. Integrating RST and semantic relationships into the proposed model also demonstrated certain advantages, allowing the generated graph structures to more accurately represent and assess text coherence. The GCN-based model also performed well, further validating the advantages of graph-based methods in capturing structural similarity and coherence. Overall, the experiments demonstrated the superiority of the GAT-based model in assessing document coherence and the effectiveness of incorporating RST and semantic relationships into graph-based methods for enhancing coherence analysis.

Table 2: Mean accuracy results on GCDC

Model	Clinton	Enron	Yahoo	Yelp	Avg
LGG	64.60	55.74	61.30	56.70	59.59
PARSEQ	60.20	53.20	54.90	54.40	55.70
Unified coherence	60.65	54.10	56.80	55.85	56.85
RST- Recursive	53.99	53.99	61.17	52.40	55.39
MTL with T	66.5	56.4	62.2	58.9	61.0
GCN-based	66.20	57.00	63.65	58.05	61.23
Our Method-GCN	68.00	58.50	**65.50**	60.30	63.08
Our Method-GAT	**69.50**	**60.25**	64.75	**62.55**	**64.26**

V. Conclusion

This paper explores how to leverage structural information between documents for coherence modelling. The proposed model constructs structural graphs based on RST and semantic relationships, identifies subgraph patterns within them, and represents the text and its subgraph patterns as nodes, with the relationships between subgraph patterns as edges, forming a document-pattern graph. Finally, GAT is employed to model this graph. The experimental results demonstrate that the proposed method performs excellently across all datasets, proving the effectiveness of integrating RST and semantic relationships into graph-based methods, as well as validating the capability of GAT to enhance performance by focusing on neighborhoods' information through its attention mechanism.

Nevertheless, this study has some limitations. For instance, the constructed document-pattern graph contains two different types of nodes representing distinct coherence information. However, the current model does not differentiate between these two types of nodes, instead feeding the document-pattern graph directly into the GAT encoder. Future research will explore the use of techniques such as heterogeneous graph neural networks to enhance the model's ability to distinguish between different types of nodes, thereby improving the effectiveness of coherence analysis.

Acknowledgments

This work is supported by the Innovation Project of GUET Graduate Education.

References

[1] Parveen, D., Mesgar, M., & Strube, M. (2016, November). Generating coherent summaries of scientific articles using coherence patterns. In *Proceedings of the 2016 conference on empirical methods in natural language processing* (pp. 772-783).

[2] Xiong, H., He, Z., Wu, H., & Wang, H. (2019, July). Modeling coherence for discourse neural machine translation. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 33, No. 01, pp. 7338-7345).

[3] Ramesh, D., & Sanampudi, S. K. (2024). Coherence-based automatic short answer scoring using sentence embedding. *European Journal of Education*, e12684.

[4] Mann, W. C., & Thompson, S. A. (1988). RST: Toward a functional theory of text organization. *Text-interdisciplinary Journal for the Study of Discourse*, *8*(3), 243-281.

[5] Li, J., & Jurafsky, D. (2016). Neural net models for open-domain discourse coherence. *arXiv preprint arXiv:1606.01545*.

[6] Mesgar, M., & Strube, M. (2016, June). Lexical coherence graph modeling using word embeddings. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 1414-1423).

[7] Lai, A., & Tetreault, J. (2018). Discourse coherence in the wild: A dataset, evaluation and methods. *arXiv preprint arXiv:1805.04993*.

[8] Moon, H. C., Mohiuddin, T., Joty, S., & Chi, X. (2019). A unified neural coherence model. *arXiv preprint arXiv:1909.00349*.

[9] Jeon, S., & Strube, M. (2022, May). Entity-based neural local coherence modeling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 7787-7805).

[10] Abhishek, T., Rawat, D., Gupta, M., & Varma, V. (2021). Transformer models for text coherence assessment. *arXiv preprint arXiv:2109.02176*.

[11] Liu, W., Fu, X., & Strube, M. (2023). Modeling structural similarities between documents for coherence assessment with graph convolutional networks. *arXiv preprint arXiv:2306.06472*.

[12] Yasunaga, M., Zhang, R., Meelu, K., Pareek, A., Srinivasan, K., & Radev, D. (2017). Graph-based neural multi-document summarization. *arXiv preprint arXiv:1706.06681*.

[13] Yao, L., Mao, C., & Luo, Y. (2019, July). Graph convolutional networks for text classification. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 33, No. 01, pp. 7370-7377).

[14] Ji, Y., & Eisenstein, J. (2014, June). Representation learning for text-level discourse parsing. In *Proceedings of the 52nd annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 13-24).

[15] Hirao, T., Yoshida, Y., Nishino, M., Yasuda, N., & Nagata, M. (2013, October). Single-document summarization as a tree knapsack problem. In *Proceedings of the 2013 conference on empirical methods in natural language processing* (pp. 1515-1520).

[16] Zhao, Q., Niu, J., Liu, X., He, W., & Tang, S. (2023). Generation of coherent multi-sentence texts with a coherence mechanism. *Computer Speech & Language*, *78*, 101457.

[17] Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. *stat*, *1050*(20), 10-48550.

[18] Guz, G., Bateni, P., Muglich, D., & Carenini, G. (2020). Neural RST-based evaluation of discourse coherence. *arXiv preprint arXiv:2009.14463*.

Authorized licensed use limited to: UNIVERSITY OF TENNESSEE LIBRARIES. Downloaded on September 23,2025 at 04:00:42 UTC from IEEE Xplore. Restrictions apply.